Approximate Dynamic Programming for Stochastic Airport Operations: A Study of Automatic State Representation Methods

Jeroen Fränzel

February 9, 2026

Abstract

This thesis investigates automatic state representation methods for Approximate Dynamic Programming (ADP) applied to stochastic airport gate assignment problems. We formulate the gate assignment problem as a finite-horizon Markov Decision Process (MDP) that explicitly incorporates schedule-based arrivals with stochastic delays and spatially heterogeneous service times constrained by runway-gate topology. We generate state-transition trajectories via simulation to construct a state-transition graph that captures the intrinsic geometry of operational bottlenecks. We study three approaches to basis function construction for linear value function approximation: handcrafted domain features, Proto-Value Functions (PVFs) derived from the spectral decomposition of the state graph, and Successor Features that predict future resource congestion. Our analysis focuses on approximation quality, interpretability, and the ability of learned representations to capture complex spatiotemporal dependencies under varying congestion and disruption scenarios. This work contributes a rigorous comparison of geometric versus engineered representation learning methods in a safety-critical operational domain where interpretability and mathematical transparency are essential.

Contents

1 Introduction **3**
- 1.1 Motivation . . . 3
- 1.2 Problem Statement . . . 3
- 1.3 Approach . . . 4
- 1.4 Contributions . . . 4
- 1.5 Thesis Outline . . . 5

2 Background **6**
- 2.1 Markov Decision Processes . . . 6
- 2.2 Approximate Dynamic Programming . . . 6
  - 2.2.1 Linear Value Function Approximation . . . 7
  - 2.2.2 Temporal Difference Learning . . . 7
- 2.3 Proto-Value Functions . . . 7
  - 2.3.1 Graph Laplacian . . . 7
  - 2.3.2 Spectral Basis Functions . . . 7
  - 2.3.3 Graph Construction for Directed Processes . . . 8
- 2.4 Successor Features . . . 8
  - 2.4.1 Utility for Transfer Learning . . . 8
- 2.5 Related Work . . . 8

3 MDP Formulation of Gate Assignment **9**
- 3.1 Problem Description . . . 9
- 3.2 State Space . . . 9
- 3.3 Action Space . . . 9
- 3.4 Transition Dynamics . . . 10
  - 3.4.1 Deterministic Time Evolution . . . 10
  - 3.4.2 Gate Occupancy Updates (Heterogeneous Service) . . . 10
  - 3.4.3 Stochastic Schedule-Based Arrivals . . . 10
  - 3.4.4 Queue Updates . . . 10
- 3.5 Reward Function . . . 10
- 3.6 Assumptions . . . 10
- 3.7 Complexity Analysis . . . 11

4 Trajectory Generation via Simulation **12**

5 State Representation Methods **13**

6 Approximate Dynamic Programming Implementation **14**

7 Experimental Design **15**

8 Results and Analysis **16**

9 Conclusion and Future Work **17**

A Mathematical Proofs **18**

B Implementation Details **19**

Chapter 1

Introduction

1.1 Motivation

Airport operations face increasing complexity due to growing air traffic volumes, resource constraints, and operational uncertainties. Gate assignment, the problem of allocating arriving aircraft to terminal gates is a critical scheduling task that directly affects passenger experience, airline efficiency, and airport throughput.

Traditional approaches to gate assignment often rely on deterministic optimization or heuristic rules, assuming fixed schedules and homogeneous resources. However, real-world operations involve significant stochastic elements and physical constraints:

- **Uncertain Arrivals:** Flights rarely arrive exactly at their scheduled times due to weather, air traffic control holds, and upstream delays.
- **Heterogeneous Resources:** Gates are not identical; varying distances between multiple runways and terminal gates create significant differences in taxiing times and total gate occupancy durations.
- **Complex Constraints:** Assessing the "cost" of an assignment requires balancing immediate taxiing penalties against future gate availability and queue overflows.

These uncertainties and geometric constraints motivate the use of stochastic sequential decision-making frameworks that can look beyond immediate greedy assignments to optimize long-term operational stability.

1.2 Problem Statement

We address the following research question:

> *How can we automatically construct effective state representations for Approximate Dynamic Programming in stochastic airport gate assignment, and can these learned representations capture the complex spatiotemporal structure of schedule-based operations better than handcrafted heuristics?*

Specifically, we investigate whether spectral graph theory can automatically identify "bottleneck" situations in airport operations—such as the interplay between specific runway arrivals and gate congestion—without explicitly programming these relationships.

1.3 Approach

Our methodology consists of the following stages:

1. **MDP Formulation**: We define the gate assignment problem as a discrete-time, finite-horizon Markov Decision Process (MDP). Unlike generic queueing models, our formulation explicitly incorporates:
   - **Schedule-based Arrivals:** Arrivals follow a master schedule with stochastic deviations (noise), rather than a pure Poisson process.
   - **Physical Topology:** Service times include variable taxiing penalties based on the distance between the assigned runway and the specific gate (heterogeneous service times).
2. **Trajectory Generation**: We simulate episodes using representative daily schedules under baseline policies (e.g., random, greedy) to collect state-transition data. This captures the "interaction dynamics" of the airport under realistic loads.
3. **Graph Construction**: We construct a weighted **state-transition graph** from the observed trajectories. *Note:* This graph represents the abstract "space of operational situations" (states), not the physical map of the airport. Nodes represent unique configurations of queues and gate occupancies, and edges represent possible transitions between them.
4. **Representation Learning**: We construct basis functions to approximate the Value Function via:
   - **Proto-Value Functions (PVFs):** Eigenvectors of the graph Laplacian, which capture the global geometry and bottlenecks of the state space.
   - **Successor Features:** Representations based on the expected future occupancy of specific state features (e.g., predicting future congestion for a specific runway).
   - **Handcrafted Features:** Standard domain-specific engineering (e.g., "number of free gates") serving as a baseline.
5. **ADP with Linear VFA**: We apply Temporal Difference (TD) learning to estimate the value of each state using the learned representations, effectively teaching the agent to recognize and avoid "bad" geometric configurations.
6. **Evaluation**: We test the learned policies under varying congestion levels and disruption scenarios (e.g., runway closures), measuring approximation quality, throughput, and robustness.

1.4 Contributions

- A rigorous MDP formulation of gate assignment that bridges the gap between abstract queueing theory and realistic airport constraints (schedules, runway distances).
- An empirical evaluation of spectral geometric methods (PVFs) in a high-dimensional resource allocation domain.
- A comparative analysis of "interpretable" automatic features vs. human-engineered heuristics for safety-critical operations.

- An open-source simulation framework enabling reproducible research in stochastic airport operations.

1.5 Thesis Outline

The remainder of this thesis is organized as follows:

- **Chapter 2**: Background on MDPs, ADP, and spectral graph theory.
- **Chapter 3**: Formal MDP formulation, including the schedule-based arrival process and runway-dependent transition dynamics.
- **Chapter 4**: Methodology for trajectory generation and handling disjoint state spaces.
- **Chapter 5**: Derivation of Proto-Value Functions and Successor Features for the gate assignment graph.
- **Chapter 6**: Implementation of the Approximate Dynamic Programming algorithm.
- **Chapter 7**: Experimental design and scenario definitions.
- **Chapter 8**: Analysis of policy performance and feature interpretability.
- **Chapter 9**: Conclusions and directions for future work.

Chapter 2

Background

2.1 Markov Decision Processes

Definition 2.1 (Finite-Horizon MDP)**.** A finite-horizon Markov Decision Process is a tuple $(\mathcal{S}, \mathcal{A}, P, R, T)$ where:

- $\mathcal{S}$ is a potentially high-dimensional state space (including continuous or discrete components).
- $\mathcal{A}$ is a finite action space.
- $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the transition probability function.
- $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function.
- $T \in \mathbb{N}$ is the finite time horizon.

A policy $\pi : \mathcal{S} \times \{0, \ldots, T-1\} \to \mathcal{A}$ maps states and time to actions. The value function under policy π is defined as the expected cumulative reward:

$$V_t^\pi(s) = \mathbb{E}^\pi \left[\sum_{k=t}^{T-1} \gamma^{k-t} R(s_k, a_k) \mid s_t = s \right] \tag{2.1}$$

where $\gamma \in [0, 1]$ is a discount factor (often $\gamma = 1$ for pure finite-horizon problems).

The optimal value function satisfies the Bellman optimality equation:

$$V_t^*(s) = \max_{a \in \mathcal{A}} \left\{ R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V_{t+1}^*(s') \right\} \tag{2.2}$$

2.2 Approximate Dynamic Programming

When the state space is large (e.g., exponential in the number of gates and queue size), exact dynamic programming becomes computationally intractable due to the "curse of dimensionality". Approximate Dynamic Programming (ADP) addresses this by replacing the exact value table with a parametric approximation.

2.2.1 Linear Value Function Approximation

We approximate the value function as a linear combination of basis functions:

$$\hat{V}(s;\theta) = \sum_{i=1}^{k} \theta_i \phi_i(s) = \phi(s)^\top \theta \tag{2.3}$$

where $\phi : \mathcal{S} \to \mathbb{R}^k$ maps states to a feature vector and $\theta \in \mathbb{R}^k$ are learnable weights. The choice of basis functions $\phi(s)$ is critical to the success of the approximation.

2.2.2 Temporal Difference Learning

To learn the weights θ, we employ Temporal Difference (TD) learning. The TD(0) update rule for linear VFA is:

$$\theta \leftarrow \theta + \alpha \left[r + \gamma \hat{V}(s';\theta) - \hat{V}(s;\theta) \right] \phi(s) \tag{2.4}$$

where $\alpha > 0$ is the learning rate and $r + \gamma \hat{V}(s';\theta)$ serves as the target value.

2.3 Proto-Value Functions

Proto-Value Functions (PVFs) provide a task-independent basis for value function approximation derived from the intrinsic geometry of the state space. They generalize Fourier analysis to irregular graphs, capturing global smooth structures (clusters, bottlenecks) that local features might miss.

2.3.1 Graph Laplacian

Given a weighted undirected graph $G = (V, E, W)$ representing the state space topology:

$$L = D - W \tag{2.5}$$

where W is the weighted adjacency matrix and D is the diagonal degree matrix with $D_{ii} = \sum_j W_{ij}$. The normalized Laplacian is defined as:

$$\mathcal{L} = I - D^{-1/2} W D^{-1/2} \tag{2.6}$$

2.3.2 Spectral Basis Functions

The eigenvectors of $\mathcal{L}$ serve as the basis functions. Let $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_{|\mathcal{S}|}$ be the eigenvalues with corresponding eigenvectors $v_1, \ldots, v_{|\mathcal{S}|}$. The first k non-trivial eigenvectors (those corresponding to the smallest non-zero eigenvalues) form the PVF basis:

$$\phi_i(s) = v_{i+1}(s), \quad i = 1, \ldots, k \tag{2.7}$$

These functions are "smooth" on the graph, meaning values vary slowly between connected states, making them ideal for approximating value functions that exhibit similar smoothness.

2.3.3 Graph Construction for Directed Processes

A critical challenge in applying PVFs to finite-horizon MDPs is the directed nature of time. A naive state-transition graph where edges follow $t \to t+1$ yields a Directed Acyclic Graph (DAG). Symmetrizing a DAG often results in trivial eigenvectors that simply index time rather than capturing operational geometry.

To address this, we construct the graph using one of two methods:

1. **Time-Collapsed Graph:** We define nodes as unique resource configurations $(\mathbf{g}, \mathbf{q})$ independent of absolute time. Edges represent valid transitions between configurations, allowing the discovery of cyclic structures and recurrent operational states.

2. **Kernel-based Adjacency:** We define edge weights based on state similarity rather than pure transitions. For states s_i, s_j, the weight W_{ij} is given by a kernel function (e.g., Gaussian kernel on queue lengths and gate occupancy), ensuring a connected topology that reflects functional similarity.

2.4 Successor Features

Successor Features (SFs) offer an alternative representation based on predictive dynamics. They generalize the Successor Representation to high-dimensional function approximation.

Definition 2.2 (Successor Features)**.** Given a feature map $\phi(s) \in \mathbb{R}^d$, the successor features $\psi^\pi(s)$ under policy π are the expected discounted future accumulation of feature observations:

$$\psi^\pi(s) = \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^t \phi(s_t) \mid s_0 = s \right] \tag{2.8}$$

If the reward function can be approximated linearly as $R(s) \approx \phi(s)^\top \mathbf{w}$, then the value function can be computed exactly as:

$$V^\pi(s) = \psi^\pi(s)^\top \mathbf{w} \tag{2.9}$$

2.4.1 Utility for Transfer Learning

SFs decouple the environment's dynamics (captured in ψ^π) from the task goals (captured in $\mathbf{w}$). In the context of airport operations, this allows an agent to learn the "physics" of gate congestion and runway usage once (via ψ^π) and rapidly adapt to changing operational priorities (e.g., shifting focus from minimizing delay to maximizing gate utilization) by simply updating the weight vector $\mathbf{w}$ without retraining the entire representation.

2.5 Related Work

[Summary of relevant literature on airport scheduling, ADP applications in operations research, spectral methods in RL, etc.]

Chapter 3

MDP Formulation of Gate Assignment

3.1 Problem Description

An airport has G gates available for aircraft turnaround operations and R runways for arrivals. Flights arrive according to a stochastic schedule over a finite time horizon T. Each flight lands on a specific runway and requires assignment to a gate for a duration that depends on both the aircraft type and the taxiing distance between the runway and the assigned gate. The objective is to minimize waiting times and queue overflows while maximizing throughput.

3.2 State Space

The state at time t is a tuple $s_t = (t, \mathbf{g}_t, \mathbf{q}_t)$ where:

- $t \in \{0, 1, \ldots, T\}$: current time index.
- $\mathbf{g}_t \in \{0, 1, \ldots, S_{\max}\}^G$: gate occupancy vector.
  - $g_i = 0$: gate i is available.
  - $g_i = s > 0$: gate i occupied, s time steps remaining.
- $\mathbf{q}_t$: queue of waiting flights.
  - Each flight f is a tuple $(T_f^{\text{arr}}, r_f, \text{type}_f, p_f)$, where:
    - T_f^{arr}: Actual arrival time.
    - $r_f \in \{1, \ldots, R\}$: The runway the flight landed on.
    - type_f: Aircraft type (determining base service time).
    - p_f: Priority class.

3.3 Action Space

At each time step, the decision-maker selects an action $a_t \in \mathcal{A}_t$ where:

$$\mathcal{A}_t = \{(f, g) : f \in \mathbf{q}_t, g \in \text{Available}(\mathbf{g}_t), \text{Compatible}(f, g)\} \cup \{\text{wait}\} \tag{3.1}$$

An action (f, g) assigns waiting flight f to available gate g. The "wait" action defers assignment to the next time step.

3.4 Transition Dynamics

The transition from s_t to s_{t+1} involves:

3.4.1 Deterministic Time Evolution

$$t \to t + 1 \tag{3.2}$$

3.4.2 Gate Occupancy Updates (Heterogeneous Service)

Gate occupancy is determined by the aircraft's base turnaround time plus a penalty for taxiing distance. Let $\Delta(r, g)$ be a deterministic distance matrix representing the taxiing time from runway r to gate g.

For each gate i:

$$g_i^{t+1} = \begin{cases} s - 1 & \text{if } g_i^t = s > 1 \\ 0 & \text{if } g_i^t = 1 \text{ (service completes)} \\ \text{BaseService}(\text{type}_f) + \Delta(r_f, i) & \text{if action assigns flight } f \text{ to gate } i \end{cases} \tag{3.3}$$

where $\text{BaseService}(\text{type}_f)$ is a random variable sampled from the distribution specific to the aircraft type.

3.4.3 Stochastic Schedule-Based Arrivals

Arrivals are modeled as a perturbed schedule rather than a pure Poisson process.
Let $\mathcal{T} = \{T_1^{\text{sched}}, T_2^{\text{sched}}, \ldots, T_N^{\text{sched}}\}$ be the master schedule of flight arrival times. The actual arrival time A_k for the k-th flight is:

$$A_k = T_k^{\text{sched}} + \delta_k \tag{3.4}$$

where δ_k is a stochastic noise term (e.g., $\delta_k \sim \mathcal{N}(0, \sigma^2)$ or an Erlang delay distribution).

3.4.4 Queue Updates

$$\mathbf{q}_{t+1} = (\mathbf{q}_t \setminus \{f_{\text{assigned}}\}) \cup \{k : A_k = t + 1\} \tag{3.5}$$

3.5 Reward Function

The immediate reward penalizes waiting and encourages efficient assignment:

$$R(s_t, a_t) = -c_{\text{wait}} \cdot |\mathbf{q}_t| - c_{\text{overflow}} \cdot \mathbb{I}[|\mathbf{q}_t| > Q_{\max}] + c_{\text{assign}} \cdot \mathbb{I}[\text{assignment made}] \tag{3.6}$$

3.6 Assumptions

Assumption 3.1. Base service times are independent random variables with finite support, dependent on aircraft type.

Assumption 3.2. The runway r_f for each flight is predetermined by external air traffic control and is observed upon arrival.

Assumption 3.3. Gates are heterogeneous with respect to accessibility. The total gate occupation time includes a taxiing penalty $\Delta(r, g)$ which makes certain gate-runway combinations strictly more costly than others.

3.7 Complexity Analysis

The state space size is:

$$|\mathcal{S}| \approx T \cdot (S_{\max} + \max(\Delta))^G \cdot \binom{F_{\max} + Q_{\max}}{Q_{\max}} \tag{3.7}$$

The introduction of the distance matrix Δ increases the effective support of the service time distribution, but does not increase the dimensionality of the state space, preserving the feasibility of the ADP approach.

Chapter 4

Trajectory Generation via Simulation

[To be developed: baseline policies, simulation algorithm, data collection]

Chapter 5

State Representation Methods

[To be developed: PVF computation, successor features, handcrafted features]

Chapter 6

Approximate Dynamic Programming Implementation

[To be developed: TD learning with linear VFA, convergence properties]

Chapter 7

Experimental Design

[To be developed: parameter settings, evaluation metrics, scenarios]

Chapter 8

Results and Analysis

[To be developed: comparative performance, interpretation, scalability]

Chapter 9

Conclusion and Future Work

[To be developed: summary of contributions, limitations, extensions]

Appendix A

Mathematical Proofs

[To be developed: detailed derivations, convergence proofs]

Appendix B

Implementation Details

[To be developed: algorithms in pseudocode, computational considerations]